

LAPORAN PRAKTIKUM 1
IoT DAN MIKROKONTROLER Teknologi IoT



Oleh:

Nama : Ariqa Andini

Npm : 25782062

Kelas : 3C

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

PRODI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

Tujuan Praktikum

- Memahami cara kerja pengoperasian *Digital Output* (mengendalikan LED) dan *Digital Input* (membaca status *push button* menggunakan rangkaian resistor *pull-down* eksternal) pada mikrokontroler.
- Menerapkan pemetaan pin (*pinout*) serta standar tegangan operasi 3.3V pada *board* mikrokontroler.
- Menyelesaikan Tugas Mandiri berupa modifikasi sistem sakelar *toggle (latching)* yang dilengkapi teknik proteksi pantulan mekanis (*debouncing*).

3. Alat dan Bahan

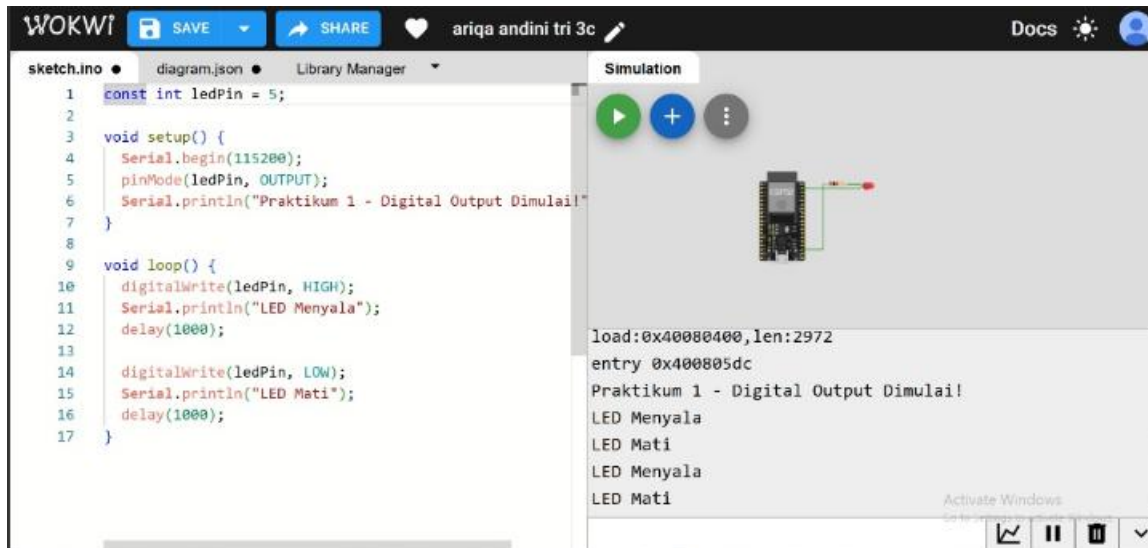
- Software Simulasi: Wokwi (Online ESP32 Simulator)
- Perangkat Keras Utama: *Board* ESP32 DevKit / NodeMCU
- Komponen Pendukung:
 - 1 buah *Tactile Push Button* (Tombol Tekan)
 - 1 buah LED 5mm
 - 1 buah Resistor 220 Ohm (pembatas arus LED)
 - Kabel *Jumper* secukupnya

4. Langkah-Langkah Percobaan & Tugas

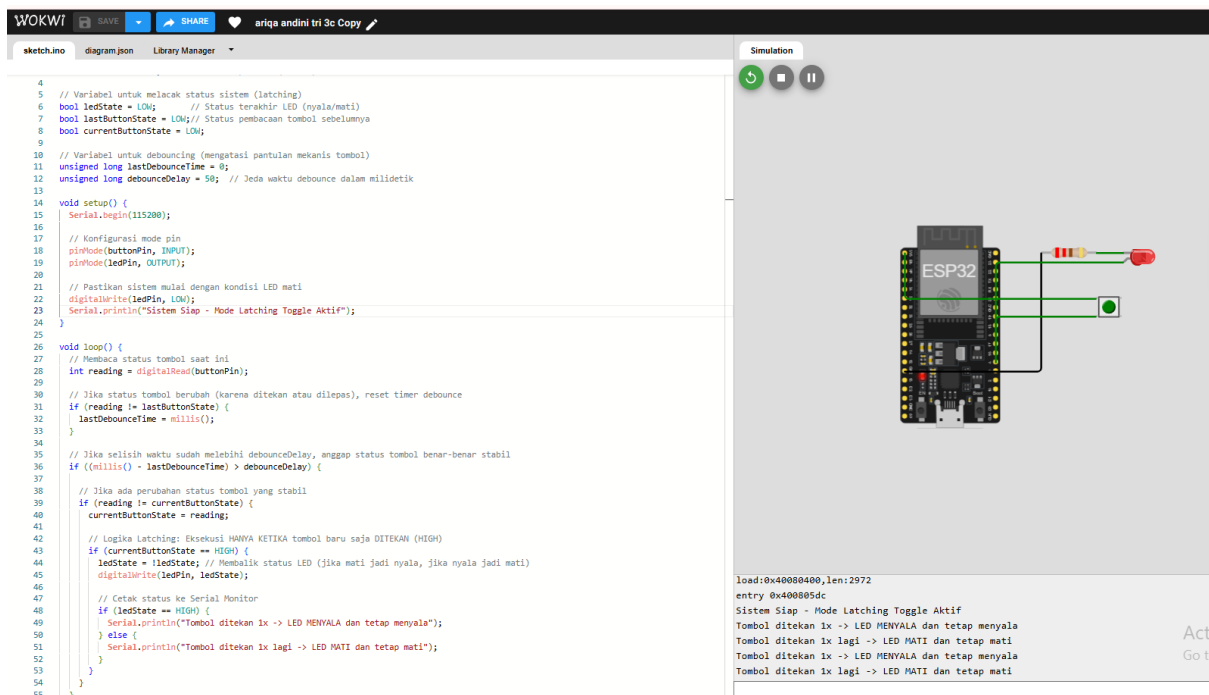
- 1. Membuka platform simulasi Wokwi dan menyiapkan proyek baru untuk *board* mikrokontroler (ESP32).**
- 2. Merangkai komponen *Digital Output* dengan menghubungkan kaki Anoda LED melalui resistor 220 Ohm ke pin digital GPIO 5 (D1), dan kaki Katoda ke jalur GND.**
- 3. Merangkai komponen *Digital Input* dengan menghubungkan salah satu kaki *push button* ke pin digital GPIO 4 (D2) dan kaki sisi lainnya langsung ke jalur GND.**
- 4. Memasukkan kode program penugasan *latching toggle* (menggunakan mode tombol yang sesuai) ke dalam editor *sketch*.**
- 5. Menjalankan simulasi, menguji fungsionalitas tombol, dan mengamati keluaran pada *Serial Monitor*.**

PRAKTIKUM

HASIL PRAKTIKUM 1



HASIL PRAKTIKIM 2



1.6 Latihan

- **Latihan 1 (Pemahaman Logika Pin):**

- **Pertanyaan:** Berdasarkan tabel Referensi Pinout di subbab 1.3.4, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2, namun menyambungkan tombol tersebut ke pin **D8 (GPIO 15)**, lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?
- **Jawaban:** Perangkat mikrokontroler dipastikan akan mengalami kegagalan proses *booting* atau *stuck* di awal.
- **Alasan:** Berdasarkan karakteristik pin ESP32, pin GPIO 15 (D8) memiliki fungsi khusus sebagai jalur konfigurasi *boot*. Berdasarkan aturannya, pin ini wajib berada dalam status logika *LOW* ketika *board* pertama kali menerima arus listrik. Jika ditekan bersamaan dengan saat perangkat dinyalakan, sistem mendeteksinya sebagai sinyal *HIGH* dan menganggapnya sebagai perintah untuk masuk ke *flash mode* alih-alih menjalankan program utama.

- **Latihan 2 (Implementasi Kode - Non-symmetric Delay):**

- **Pertanyaan:** Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris* (nyala cepat 200 ms, mati lama 800 ms).
- **Jawaban:** Dua baris fungsi `delay()` yang diubah adalah:

C++

```
digitalWrite(ledPin, HIGH);
```

```
Serial.println("LED Menyala");
```

```
delay(200); // Diubah menjadi 200 ms agar durasi nyala sangat cepat
```

```
digitalWrite(ledPin, LOW);
```

```
Serial.println("LED Mati");
```

```
delay(800); // Diubah menjadi 800 ms agar durasi mati lebih lama
```

- **Latihan 3 (Analisis Sirkuit - Polaritas LED Terbalik):**

- **Pertanyaan:** Apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program *error*? Mengapa LED tidak menyala?
- **Jawaban:**

- *Status Kompilasi*: Proses *compile* dan *upload* tetap berhasil 100% tanpa *error* karena kesalahan fisik tidak memengaruhi baris kode program.
- *Kondisi Lampu*: Secara fisik, LED sama sekali tidak memancarkan cahaya (mati).
- *Analisis Fisik*: Karakteristik dioda pada LED hanya melewatkan arus satu arah (dari Anoda ke Katoda). Ketika dibalik, arus tertahan dan gagal melewati dioda sehingga lampu tidak menyala.

- **Latihan 4 (Pengembangan Algoritma - Inversi Logika Tombol):**

- **Pertanyaan:** Ubah blok kondisi *if-else* pada Praktikum 2 sehingga perilakunya terbalik ("Saat tombol TIDAK ditekan, LED menyala. Saat tombol DITEKAN, LED mati").
- **Jawaban:**

C++

```
buttonState = digitalRead(buttonPin);
```

```
if (buttonState == LOW) {
    digitalWrite(ledPin, HIGH); // Saat tombol dilepas -> LED menyala
} else {
    digitalWrite(ledPin, LOW); // Saat tombol ditekan -> LED mati
}
```

1.7 Tugas Mandiri (Modifikasi Sistem Sakelar *Toggle* / *Latching*)

- **Tujuan:** Mengimplementasikan logika pelacakan status (*state tracking*) sakelar *toggle*.
- **Kode Program (sketch.ino):**

C++

```
// Definisi Pin sesuai rangkaian Wokwi
```

```
const int buttonPin = 4; // Pin input push button (GPIO 4)
```

```
const int ledPin = 5; // Pin output LED (GPIO 5)
```

```
// Variabel untuk melacak status sistem (latching)
```

```
bool ledState = LOW; // Status terakhir LED (nyala/mati)
```

```

bool lastButtonState = LOW; // Status pembacaan tombol sebelumnya
bool currentButtonState = LOW;

// Variabel untuk debouncing (mengatasi pantulan mekanis tombol)
unsigned long lastDebounceTime = 0;
unsigned long debounceDelay = 50; // Jeda waktu debounce dalam milidetik

void setup() {
  Serial.begin(115200);

  // Konfigurasi mode pin
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);

  // Pastikan sistem mulai dengan kondisi LED mati
  digitalWrite(ledPin, LOW);
  Serial.println("Sistem Siap - Mode Latching Toggle Aktif");
}

void loop() {
  // Membaca status tombol saat ini
  int reading = digitalRead(buttonPin);

  // Jika status tombol berubah, reset timer debounce
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }

  // Jika selisih waktu sudah melebihi debounceDelay, anggap stabil

```

```

if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != currentButtonState) {
        currentButtonState = reading;

        // Logika Latching: Eksekusi HANYA KETIKA tombol baru saja DITEKAN
(HIGH)
        if (currentButtonState == HIGH) {
            ledState = !ledState; // Membalik status LED
            digitalWrite(ledPin, ledState);

            // Cetak status ke Serial Monitor
            if (ledState == HIGH) {
                Serial.println("Tombol ditekan 1x -> LED MENYALA dan tetap menyala");
            } else {
                Serial.println("Tombol ditekan 1x lagi -> LED MATI dan tetap mati");
            }
        }
    }
}

// Simpan status pembacaan untuk perbandingan di loop berikutnya
lastButtonState = reading;
}

```

- **Hasil & Pembahasan Tugas:** Berdasarkan simulasi di Wokwi, sistem berhasil beroperasi layaknya sakelar lampu ruangan. Tombol ditekan sekali lalu dilepas membuat LED menyala dan tetap menyala; ditekan sekali lagi membuat LED mati dan tetap mati. Fungsi `millis()` dan variabel `ledState` bekerja optimal meredam *flickering* akibat gangguan mekanis tombol.

1.8 Rangkuman

Pada Modul ini, kita telah meletakkan fondasi pemrograman *Internet of Things* (IoT) menggunakan NodeMCU ESP8266. Pemahaman terhadap standar tegangan operasi 3.3V dan

pemetaan pin khusus menjadi landasan penting dalam merancang sistem *Digital Input* dan *Digital Output* secara stabil dan akurat.

1.7 Tugas Mandiri (Modifikasi Sistem Sakelar *Toggle / Latching*)

- **Tujuan:** Mengimplementasikan logika pelacakan status (*state tracking*) sakelar *toggle*.
- **Kode Program (sketch.ino):**

C++

// Definisi Pin sesuai rangkaian

const int buttonPin = 4; // Pin input push button (GPIO 4 / D2)

const int ledPin = 5; // Pin output LED (GPIO 5 / D1)

// Variabel untuk melacak status sistem (latching)

bool ledState = LOW; // Status terakhir LED (nyala/mati)

int lastButtonState = LOW; // Menyimpan status pembacaan tombol sebelumnya

void setup() {

Serial.begin(115200);

// Konfigurasi mode pin

pinMode(buttonPin, INPUT);

pinMode(ledPin, OUTPUT);

// Pastikan sistem mulai dengan kondisi LED mati

digitalWrite(ledPin, LOW);

Serial.println("Sistem Siap - Mode Latching Toggle Aktif");

}

void loop() {

// Membaca status tombol saat ini

int currentButtonState = digitalRead(buttonPin);

```

// Deteksi perubahan: Jika tombol ditekan (HIGH) dari kondisi sebelumnya (LOW)
if (currentButtonState == HIGH && lastButtonState == LOW) {
    // Instruksi delay singkat (200 ms) untuk mengatasi pantulan mekanis (debouncing)
    delay(200);

    // Membalikkan status LED (Latching / Toggle Logic)
    ledState = !ledState;
    digitalWrite(ledPin, ledState);

    // Cetak status ke Serial Monitor
    if (ledState == HIGH) {
        Serial.println("Tombol ditekan 1x -> LED MENYALA dan tetap menyala");
    } else {
        Serial.println("Tombol ditekan 1x lagi -> LED MATI dan tetap mati");
    }
}

// Simpan status tombol saat ini sebagai referensi perbandingan berikutnya
lastButtonState = currentButtonState;
}

```

- **Hasil & Pembahasan Tugas:** Berdasarkan simulasi di Wokwi, sistem berhasil beroperasi layaknya sakelar lampu ruangan. Tombol ditekan sekali lalu dilepas membuat LED menyala dan tetap menyala; ditekan sekali lagi membuat LED mati dan tetap mati. Penggunaan fungsi `delay(200)` bekerja optimal meredam gangguan mekanis tombol (*debouncing*).

1.8 Rangkuman

Pada Modul ini, kita telah meletakkan fondasi pemrograman *Internet of Things* (IoT) menggunakan NodeMCU ESP8266. Pemahaman terhadap standar tegangan operasi 3.3V dan pemetaan pin khusus menjadi landasan penting dalam merancang sistem *Digital Input* dan *Digital Output* secara stabil dan akurat[cite: 1].